Line Following Autonomous Driving

1. Course Content

1. Learn about the robot's line-following autonomous driving feature.

   After starting the program, the robot will lock onto the landmark line in front of it. After pressing the spacebar, the robot will follow the ground marker line. If an obstacle is encountered along the way, it will pause and the buzzer will sound an alarm. It will stop moving when the marker line disappears.

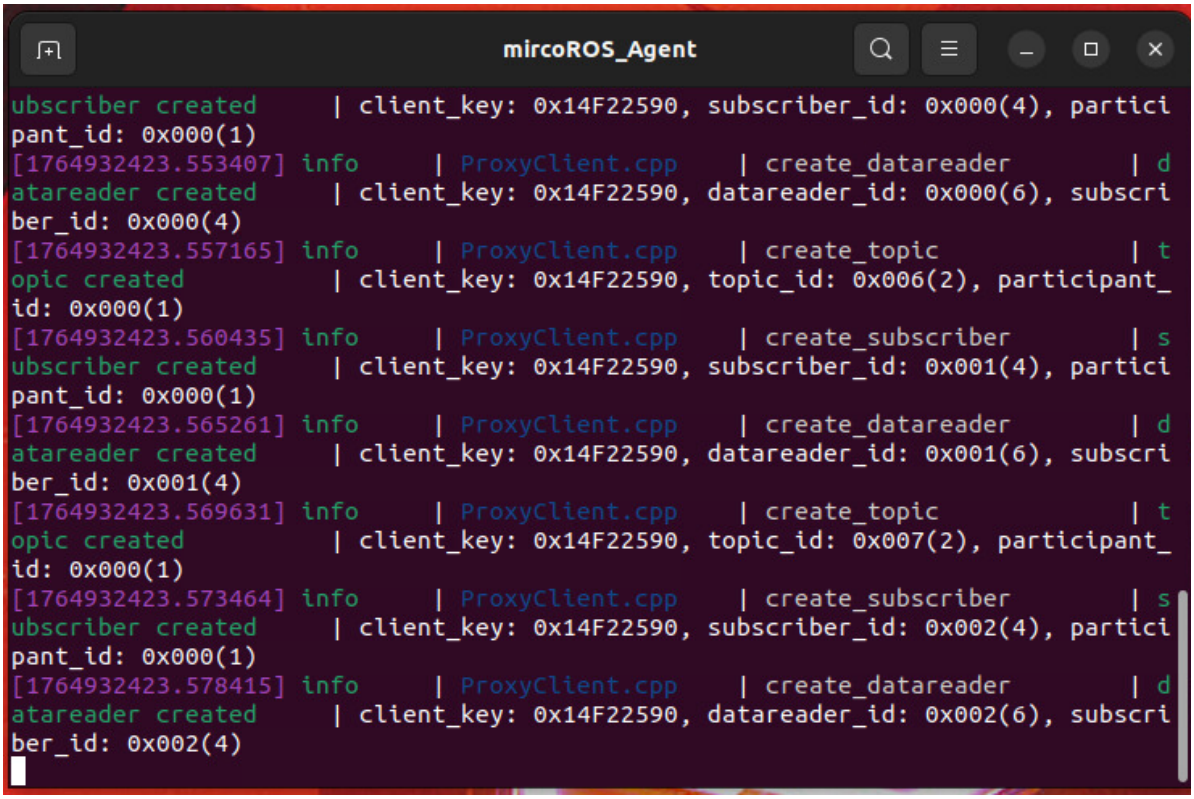
2. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:



3. Run the Example

3.1 Start the Program

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

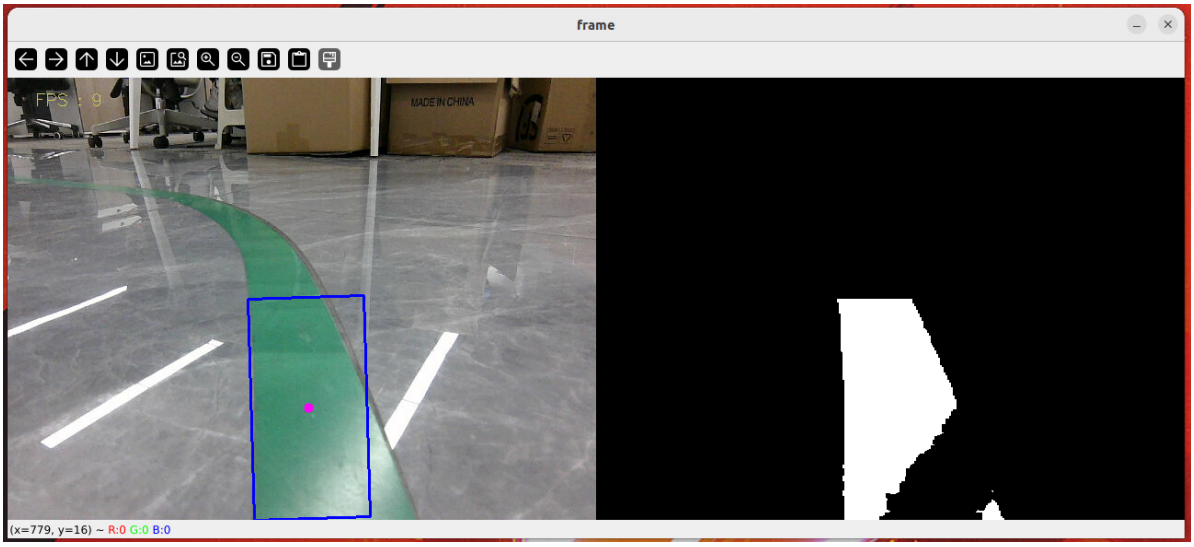
```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

- Enter the launch command in the terminal:

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=follow_line
```

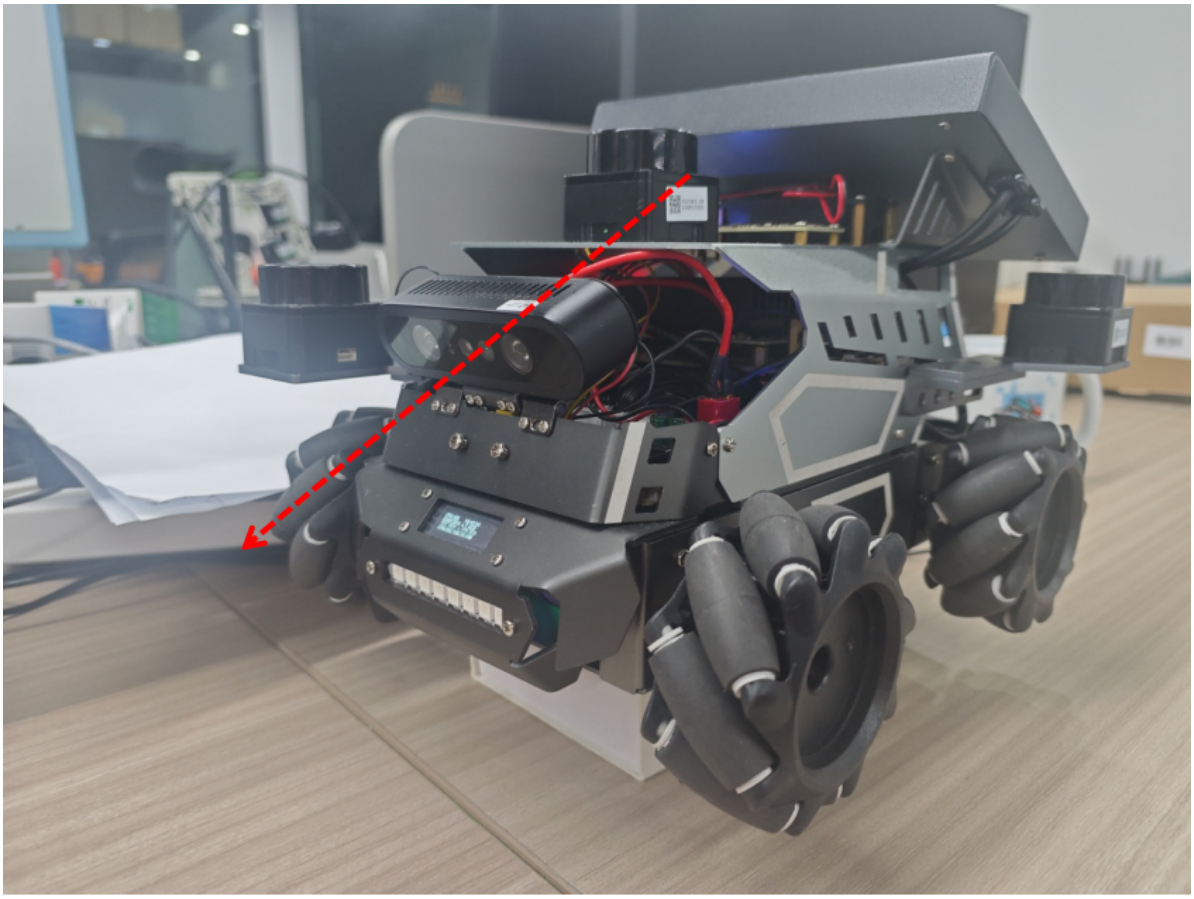


- After launching the command, a graphical window titled **frame** will open, and the marker box in the image will mark the landmark line.
- After pressing the spacebar, the robot will start moving along the marker line. If an obstacle is detected ahead, it will stop following the line and the buzzer will sound an alarm. The robot will continue moving after the obstacle is removed.
- The car will stop when the marker line ends, and the terminal will display **Not Found**, indicating that no marker line was found.

> [!NOTE]
>
> - If you see the **Not Found** prompt after pressing the spacebar, please recalibrate the HSV values before attempting to follow the line again.

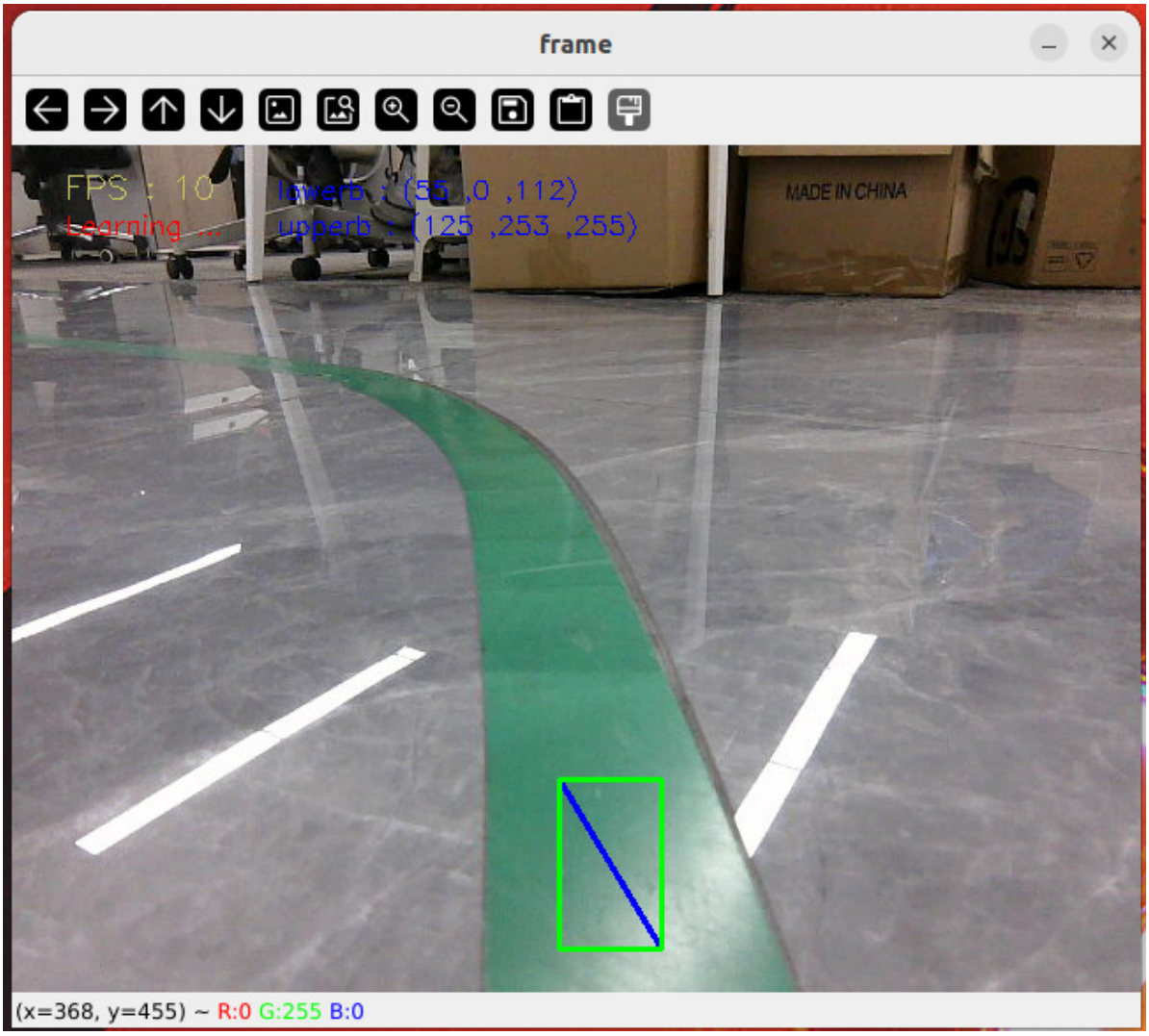
3.2 Debugging the Line-Following Effect

> [!IMPORTANT]
>
> - When running the line-following autonomous driving example, it is best to adjust the camera angle downwards slightly so that the line is clearest in the camera's view, which will improve the line-following performance.

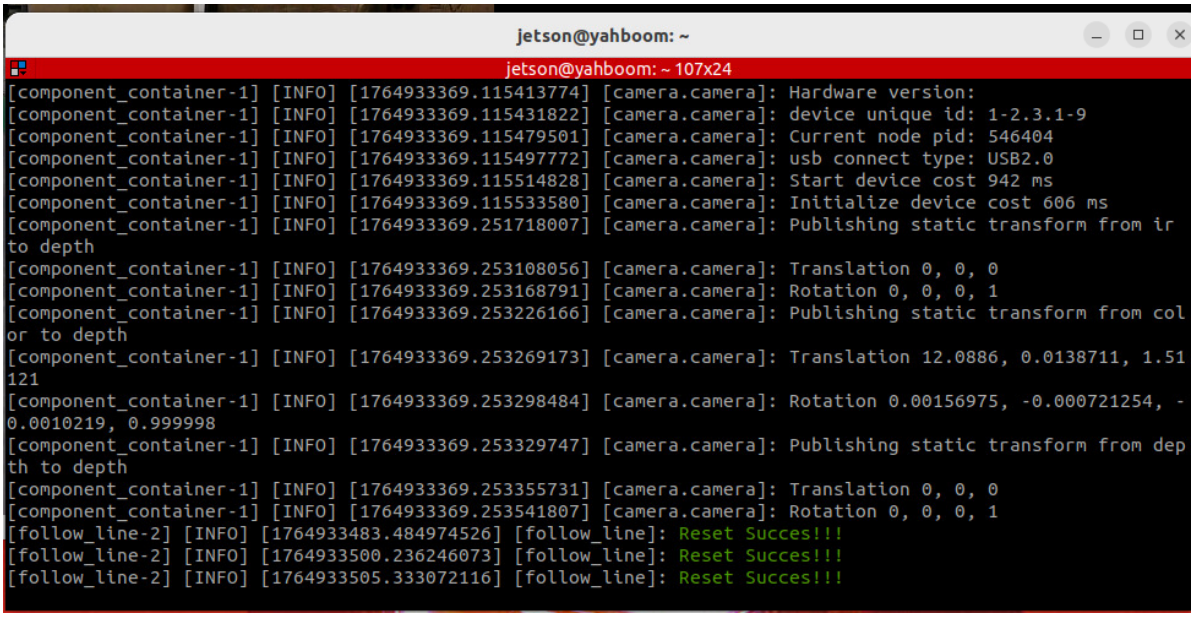


The robot is calibrated with a green `HSV` value at the factory. If the color recognition of the marker line is not ideal during line following, or if you need to change the color to follow, you will need to change the line-following color.

Press the `R` key on the keyboard to select a color. Hold down the left mouse button and drag a rectangle within the color area (making sure the rectangle is within the color range). The color will be automatically confirmed when you release the button.



After recalibrating the color, the terminal will display **Reset success!!!**, and the color calibration is complete.



4. Source Code Analysis

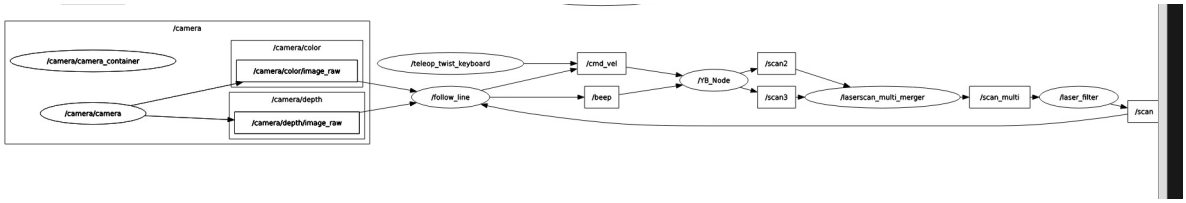
Source code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/follow_line.py
```

4.1 View Node Relationship Graph

Open a terminal and enter the command:

```
ros2 run rqt_graph rqt_graph
```



In the node relationship diagram above:

- The **follow_Line** node subscribes to the camera topic's image information for line-following decisions, subscribes to `/scan` topic data to check for obstacles ahead, and controls the buzzer by publishing to the **/beep** topic and the chassis movement by publishing to the **/cmd_vel** topic.
- The **camera** node is the camera driver node, responsible for driving the camera and publishing topic data for the RGB and depth images.

4.2 Key Program

Below is an explanation of the core parts of the program:

Image Display: Subscribes to the camera topic's image, converts it to OpenCV format, and displays it.

Program Implementation: The `callback` function in the `LineDetect` class.

```
def callback(self,color_frame,depth_frame):
    # Convert the image to OpenCV format
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame,'rgb8')
    rgb_image = np.copy(rgb_image)

    depth_image = self.depth_bridge.imgmsg_to_cv2(depth_frame, encoding[1])
    #depth_to_color_image = cv2.applyColorMap(cv2.convertScaleAbs(depth_image,
alpha=1.0), cv2.COLORMAP_JET)
    depth_img = cv2.resize(depth_image, (640, 480))
    self.depth_image_info = depth_img.astype(np.float32)

    self.tags = self.at_detector.detect(cv2.cvtColor(rgb_image,
cv2.COLOR_RGB2GRAY), False, None, 0.025)
    self.tags = sorted(self.tags, key=lambda tag: tag.tag_id)
    draw_tags(rgb_image, self.tags, corners_color=(0, 0, 255), center_color=(0,
255, 0))

    #depth_image
    depth_image = self.depth_bridge.imgmsg_to_cv2(depth_frame, encoding[1])
    #depth_to_color_image = cv2.applyColorMap(cv2.convertScaleAbs(depth_image,
alpha=1.0), cv2.COLORMAP_JET)
```

```
    frame = cv2.resize(depth_image, (640, 480))
    depth_image_info = frame.astype(np.float32)
    action = cv2.waitKey(1)
    if self.count==True and self.Start_==True:
        if (time.time() - self.start_time)>3:
            self.Track_state = 'tracking'
            self.count = False
    result_img,bin_img = self.process(rgb_image,action)
    result_img = cv2.cvtColor(result_img, cv2.COLOR_RGB2BGR)
    if len(bin_img) != 0: cv.imshow('frame', ManyImgs(1, ([result_img,
bin_img])))
    else:cv.imshow('frame', result_img)
```

Line-Following Function: Follows the marker line and stops when an obstacle is encountered.

Program Implementation: The `execute` method in the `LineDetect` class.

```
    def execute(self, point_x, color_radius):
        if self.Joy_active == True:
            b = Bool()
            self.Buzzer_state = False
            b.data = False
            self.pub_Buzzer.publish(b)
            if self.Start_state == True:
                self.PID_init()
                self.Start_state = False
            return
        self.Start_state = True
        if color_radius == 0:
            self.get_logger().info(Fore.RED + 'Not Found'+Fore.RESET)

            self.pub_cmdVel.publish(Twist())
        else:
            twist = Twist()
            b = UInt16()
            [z_Pid, _] = self.PID_controller.update([(point_x - 320)*1.0/16, 0])
            twist.angular.z = +z_Pid
            twist.linear.x = self.linear
            #print(f"twist.angular.z: {twist.angular.z}")
            if self.warning > 10:
                self.get_logger().info(Fore.YELLOW + 'Obstacles ahead
!!!'+Fore.RESET)
                self.pub_cmdVel.publish(Twist())
                self.Buzzer_state = True
                b.data = 1
                self.pub_Buzzer.publish(b)
            else:
                if self.Buzzer_state == True:
                    b.data = 0
                    for i in range(3): self.pub_Buzzer.publish(b)
                    self.Buzzer_state = False

                if abs(point_x-320)<40:
                    twist.angular.z=0.0
                if self.Joy_active == False:
                    self.pub_cmdVel.publish(twist)
                else:
```

```
                    twist.angular.z=0.0
```